

AprilTag Machine Code Follow (PTZ)

1. Experiment Objective

PTZ pan-tilt + car body collaborative AprilTag following — dt_apriltags detects the AprilTag, the 3D pose Z-axis enables precise distance following, the pan-tilt locks onto the Tag center with PD, and the body homes in coordination.

2. Experiment Principle

1. Key Terms

Term	Description
Visual Following	Pan-tilt tracking + coordinated body movement
AprilTag	A high-precision visual fiducial marker; the dt_apriltags detector supports 6-DOF pose estimation
3D pose distance following	PD distance control based on <code>pose_t[2]</code> (Z-axis depth, in meters), more precise than pixel size
Error sharing / angular-rate feedforward	The core mechanisms of the three-layer coordinated control
Lost-frame tolerance	The body is frozen only after N consecutive lost frames (default 5 frames)

2. Technical Architecture

Node chain:

State estimation layer → `april_tag_follow_ptz_node` → `/cmd_ptz_angle` (pan-tilt angle) + `/cmd_vel` (body velocity)

Three-layer coordinated control:

- **PTZ PD control (30 Hz):** Tag center pixel error → PD (`pd_kp`=1.2, `pd_kd`=0.25) → step limiting ($\pm 1.2^\circ$) → angular-rate feedforward compensation (`body_ff_gain`=1.3) → `/cmd_ptz_angle`. Deadband 50 px (larger; AprilTag detection is stable and needs no fine-tuning).
- **Body angular-velocity PD control (10 Hz):** pan-tilt horizontal angle × `body_share` (0.5) → error shared with the body → PD (`angular_kp`=0.2, `angular_kd`=0.06) → limiting ± 0.12 rad/s.
- **3D distance-follow PD control (10 Hz):** deviation between the Tag's Z-axis depth and `target_tag_pose_z` (0.35 m) → PD (`distance_kp`=0.6, `distance_kd`=0.2, the highest gains — meter-level precision) → limiting ± 0.12 m/s.

Multiple-Tag selection and visualization:

`tag_select_strategy=largest` selects the primary Tag. On-screen overlay: camera coordinate axes (X/Y/Z), the primary Tag as a purple polygon + `tag_id`, and the other Tags as green polygons.

3. Core Topics Quick Reference

Firmware Uplink (ESP32 → ROS2):

Topic Name	Type	QoS	Description
<code>/camera/image_raw</code>	sensor_msgs/Image	KEEP_LAST 1, BEST_EFFORT, VOLATILE	Raw image input from the ESP32 camera

Firmware Downlink (ROS2 → ESP32 via micro_ros_agent):

Topic Name	Type	QoS	Description
<code>/cmd_ptz_angle</code>	geometry_msgs/Vector3	Default (KEEP_LAST 10)	PTZ pan-tilt angle command (x: horizontal, y: tilt)
<code>/cmd_vel</code>	geometry_msgs/Twist	Default (KEEP_LAST 10)	Body velocity command (linear.x: forward/back, angular.z: rotation)

ROS2 Internal (published by the state estimation layer):

Topic Name	Type	QoS	Description
<code>/diagnostics</code>	diagnostic_msgs/DiagnosticArray	Default	Summary of each node's running status

3. Experiment Steps

1. Hardware Preparation

Hardware	Requirement
PTZ version	☒
No-PTZ version	☐
Required peripherals	ESP32 camera (pan-tilt) + PTZ servo + an AprilTag (0.048 m size recommended)

2. Start Agent + Camera + Joint Model (Terminal 1)

```
ros2 launch microros_v2_bringup car_base.launch.py
```

```

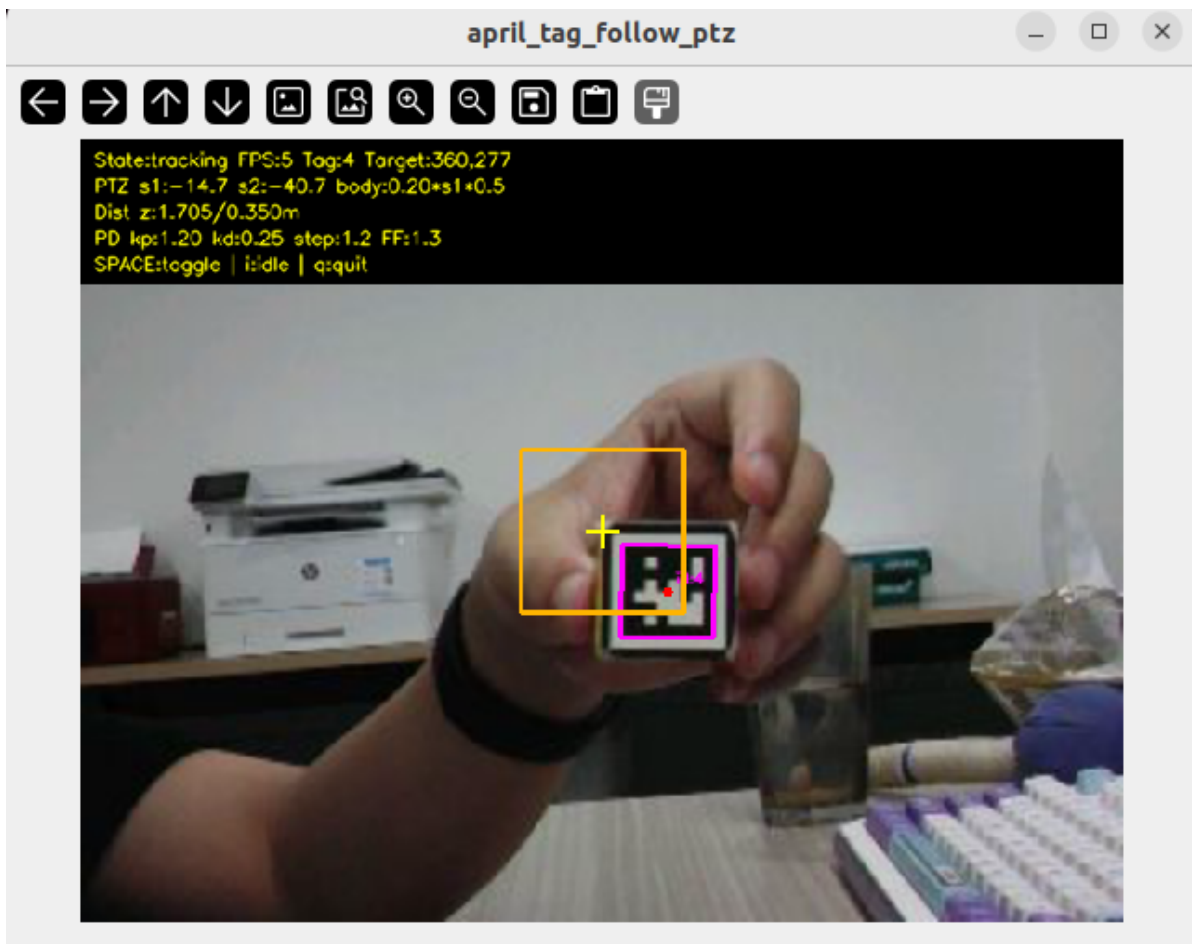
yahboom@yahboom:~$ ros2 launch microros_v2_bringup car_base.launch.py
[INFO] [launch]: All log files can be found below /home/yahboom/.ros/log/2026-07-28-15-04-46-269642-yahboom-579370
[INFO] [launch]: Default logging verbosity is set to INFO
[INFO] [micro_ros_agent-1]: process started with pid [579411]
[INFO] [camera_driver-2]: process started with pid [579413]
[INFO] [robot_state_publisher-3]: process started with pid [579415]
[INFO] [joint_state_publisher-4]: process started with pid [579417]
[INFO] [ekf_node-5]: process started with pid [579419]
[micro_ros_agent-1] [1785222287.404236] info | UDPv4AgentLinux.cpp | init | running... | port: 8090
[robot_state_publisher-3] [INFO] [1785222288.847816851] [robot_state_publisher]: got segment base_footprint
[robot_state_publisher-3] [INFO] [1785222288.848013042] [robot_state_publisher]: got segment base_link
[robot_state_publisher-3] [INFO] [1785222288.848028942] [robot_state_publisher]: got segment bwheel_Link
[robot_state_publisher-3] [INFO] [1785222288.848037122] [robot_state_publisher]: got segment cam1_Link
[robot_state_publisher-3] [INFO] [1785222288.848043874] [robot_state_publisher]: got segment cam2_Link
[robot_state_publisher-3] [INFO] [1785222288.848050538] [robot_state_publisher]: got segment cam3_Link
[robot_state_publisher-3] [INFO] [1785222288.848057264] [robot_state_publisher]: got segment imu_frame
[robot_state_publisher-3] [INFO] [1785222288.848063747] [robot_state_publisher]: got segment laser_frame
[robot_state_publisher-3] [INFO] [1785222288.848070240] [robot_state_publisher]: got segment lfwheel_Link
[robot_state_publisher-3] [INFO] [1785222288.848076888] [robot_state_publisher]: got segment rfwheel_Link
[joint_state_publisher-4] [INFO] [1785222289.372349515] [joint_state_publisher]: Waiting for robot_description to be published on the robot_description
[camera_driver-2] [INFO] [1785222289.716161193] [esp32_ip_camera_publisher]: camera node started, camera_url: http://192.168.12.37:81/stream
[camera_driver-2] [WARN] [1785222292.785741414] [esp32_ip_camera_publisher]: Camera not opened, trying reconnect... (next retry in 1.0s)
[camera_driver-2] [INFO] [1785222294.072701944] [esp32_ip_camera_publisher]: IP camera stream opened successfully

```

3. Start AprilTag Follow (Terminal 2)

Open a new terminal and run:

```
ros2 launch microros_v2_ptz_visual_control_pkg april_tag_follow_ptz.launch.py
```



After the node starts, the camera intrinsics (`camera.yaml`) are loaded automatically, and the Space key toggles the idle/tracking state. Without a calibration file, it automatically degrades to pixel-size distance following.

4. Operation Instructions

Interaction:

Key	Function
Space	idle ↔ tracking toggle
i / I	Force idle
q / Q / ESC	Exit the program

Workflow:

1. After the program starts, it is in the **idle** state by default; AprilTag detection is already running but **outputs no control commands**
2. Place the AprilTag within the camera's field of view, then press **Space** to enter the **tracking** state
3. The pan-tilt locks onto the Tag center, the body turns to home, and a distance of about 0.35 m is maintained automatically
4. Press **i** to return to idle, press **q** to exit

Safety and Edge Cases:

- **dt_apriltags unavailable**: an ImportError is logged; the node does not crash
- **Camera YAML unavailable**: pose estimation is disabled; only 2D pixel tracking and distance following are performed (degraded mode)
- No Tag ≥5 frames → freeze the body (`lost_frame_threshold=5`)
- Pan-tilt limiting: horizontal [-90°, 90°], tilt [-90°, 20°]
- Velocity limiting: angular velocity ±0.12 rad/s, linear velocity ±0.12 m/s
- `quad_decimate` ≥1.0, validated at runtime

5. How to Stop

Press `Ctrl+C` to stop Terminal 2 → Terminal 1 in order.

4. Experiment Observations

- **idle**: AprilTag detection runs but outputs no control; the frame shows the Tag border and id
- **tracking + Tag**: The pan-tilt locks onto the Tag center, the body turns to home, and 3D distance following maintains a depth of about 0.35 m
- **With camera calibration**: the pose X/Y/Z is displayed precisely, and distance control is based on the meter-level Z axis
- **Without camera calibration**: degrades to pixel-size distance following (based on the Tag area)
- **Tag moves forward/back**: the car adjusts forward/back to maintain a $0.35\text{ m} \pm 0.06\text{ m}$ depth
- **Multiple Tags**: the primary Tag is marked purple + the others green, selected by the largest strategy
- **Tag gone ≥5 frames**: the body freezes

5. Program Analysis

Source code paths:

- Launch file:
`~/microros_ws/src/microros_v2_ptz_visual_control_pkg/launch/april_tag_follow_ptz.launch.py`
- Node file:
`~/microros_ws/src/microros_v2_ptz_visual_control_pkg/microros_v2_ptz_visual_control_pkg/april_tag_follow_ptz_node.py`

1. Core Node Analysis

`AprilTagFollowPtzNode` inherits from `rcipy.node.Node`; detection and display are done in the image callback, with two independent control timers for PTZ/body. It supports dynamic parameter hot-reloading and rqt dynamic tuning.

Node initialization — `__init__`:

```
# Source file:
~/microros_ws/src/microros_v2_ptz_visual_control_pkg/microros_v2_ptz_visual_control_pkg/april_tag_follow_ptz_node.py

class AprilTagFollowPtzNode(Node):

    def __init__(self) -> None:
        super().__init__(DEFAULT_NODE_NAME)
        self._tgt_lock = threading.Lock()
        self._latest_frame: Optional[np.ndarray] = None

        self._state = STATE_IDLE
        self._shutdown = False

        self._has_target = False
        self._lost_frames = 0
        self._tag_id = -1
        self._target_x: Optional[float] = None
        self._target_y: Optional[float] = None
        self._tag_pose_x: Optional[float] = None
        self._tag_pose_y: Optional[float] = None
        self._tag_pose_z: Optional[float] = None
        self._tag_corners: List[Tuple[int, int]] = []

        self._h_deg = 0.0
        self._p_deg = 0.0
        self._prev_err_x = 0.0
        self._prev_err_y = 0.0
        self._prev_ctrl_t = 0.0
        self._last_sx = 0.0
        self._last_sy = 0.0

        self._prev_s1 = 0.0
        self._prev_s1_eff = 0.0
        self._prev_body_t = 0.0
        self._last_body_az = 0.0

        self._prev_z_err = 0.0
        self._prev_linear_t = 0.0

        self._fps = 0.0
```

```

self._fps_n = 0
self._fps_t = time.perf_counter()
self._win_ok = False
self._last_print_t = 0.0

self._detector = None
self._camera_params: Optional[Tuple[float, float, float, float]] = None

self._declare_params()
self._load_params()
self._h_deg = self._init_h_deg
self._p_deg = self._init_p_deg
self._init_detector()

video_qos = QoSProfile(
    history=QoSHistoryPolicy.KEEP_LAST,
    depth=1,
    reliability=QoSReliabilityPolicy.BEST_EFFORT,
    durability=QoSDurabilityPolicy.VOLATILE,
)

self._sub = self.create_subscription(
    Image, self._img_topic, self._on_image, video_qos
)
self._ptz_pub = self.create_publisher(Vector3, self._ptz_topic, 10)
self._cmd_pub = self.create_publisher(Twist, self._cmd_topic, 10)

self._ptz_timer = self.create_timer(
    1.0 / max(self._ptz_rate, MINIMUM_TIMER_RATE_HZ), self._ptz_tick
)
self._body_timer = None
self._rebuild_body_timer(False)
self._key_timer = None
if self._show_win:
    self._init_win()
    self._key_timer = self.create_timer(0.05, self._poll_keys)

self._init_ptz_timer = None
self._init_ptz_remain = 0
if self._pub_init:
    self._start_init_ptz()

self._param_cb =
self.add_on_set_parameters_callback(self._on_param_update)
self.get_logger().info(
    f"AprilTag follow PTZ+body started. SPACE=toggle, i=idle, q=quit. "
    f"ptz={self._ptz_rate:.0f}Hz body={self._body_rate:.0f}Hz "
    f"family={self._tag_family} tag_size={self._tag_size:.3f}m"
)

```

Distance-follow core — `_body_tick` (3D Z-axis PD control):

```

# Source file:
~/microros_ws/src/microros_v2_ptz_visual_control_pkg/microros_v2_ptz_visual_control_pkg/april_tag_follow_ptz_node.py

def _body_tick(self) -> None:

```

```

if self._state != STATE_TRACKING:
    self._prev_s1 = 0.0
    self._prev_s1_eff = 0.0
    self._prev_body_t = 0.0
    self._pub_zero_cmd()
    return

with self._tgt_lock:
    ok = self._has_target
    pose_z = self._tag_pose_z
if not ok:
    self._prev_s1 = 0.0
    self._prev_s1_eff = 0.0
    self._prev_body_t = 0.0
    self._pub_zero_cmd()
    return

s1 = self._h_deg
if abs(s1) < self._ang_tol:
    s1 = 0.0
s1_eff = s1 * self._body_share

n = time.perf_counter()
dt = n - self._prev_body_t if self._prev_body_t > 0.0 else 1.0 /
max(self._body_rate, MINIMUM_TIMER_RATE_HZ)
dt = max(dt, 1e-3)
ds1_eff = (s1_eff - self._prev_s1_eff) / dt
az = self._ang_kp * s1_eff + self._ang_kd * ds1_eff
self._prev_s1 = s1
self._prev_s1_eff = s1_eff
self._prev_body_t = n

if self._rev_body:
    az = -az
az = clamp(az, -self._max_az, self._max_az)
self._last_body_az = az

if self._enable_dist and pose_z is not None:
    z_err = pose_z - self._target_pose_z
    if abs(z_err) < self._dist_tol_z:
        z_err = 0.0
    dt_l = n - self._prev_linear_t if self._prev_linear_t > 0.0 else 1.0
/ max(self._body_rate, MINIMUM_TIMER_RATE_HZ)
dt_l = max(dt_l, 1e-3)
dz = (z_err - self._prev_z_err) / dt_l
lx = self._dist_kp * z_err + self._dist_kd * dz
self._prev_z_err = z_err
self._prev_linear_t = n
if self._rev_linear:
    lx = -lx
lx = clamp(lx, -self._max_lx, self._max_lx)
else:
    self._prev_z_err = 0.0
    lx = 0.0

tw = Twist()
tw.angular.z = float(az)
tw.linear.x = float(lx)

```

```
self._cmd_pub.publish(tw)
```

The 3D distance-follow key line: `z_err = pose_z - self._target_pose_z` — the `pose_t[2]` (Z-axis depth, in meters) returned by the AprilTag pose estimation is used as the error input, and after PD computation the `linear.x` velocity is output. The `distance_kp=0.6` / `distance_kd=0.2` gains are significantly higher than the pixel-size scheme, because the meter-level error range is larger.

2. Key Parameters

Parameter definition files:

- Launch file:
`~/microros_ws/src/microros_v2_ptz_visual_control_pkg/launch/april_tag_follow_ptz.launch.py`
- Node file:
`~/microros_ws/src/microros_v2_ptz_visual_control_pkg/microros_v2_ptz_visual_control_pkg/april_tag_follow_ptz_node.py`

Pan-tilt PTZ control parameters:

Parameter	Type	Default Value	Description
<code>pd_kp</code>	float	1.2	Pan-tilt PD proportional gain
<code>pd_kd</code>	float	0.25	Pan-tilt PD derivative gain
<code>ptz_control_rate</code>	float	30.0 Hz	Pan-tilt control rate (highest)
<code>deadband_x</code>	float	50.0 px	Horizontal deadband (larger; Tag detection is stable and needs no fine-tuning)
<code>deadband_y</code>	float	50.0 px	Vertical deadband
<code>max_ptz_step</code>	float	1.2°	Maximum single step angle
<code>ptz_step_smooth_alpha</code>	float	0.35	Step smoothing coefficient
<code>ptz_smooth_alpha</code>	float	0.85	Angle output smoothing coefficient

Body control parameters:

Parameter	Type	Default Value	Description
<code>body_control_rate</code>	float	10.0 Hz	Body control rate
<code>angular_kp</code>	float	0.2	Body angular-velocity P gain
<code>angular_kd</code>	float	0.06	Body angular-velocity D gain
<code>body_share</code>	float	0.5	Ratio of the pan-tilt error allocated to the body
<code>body_ff_gain</code>	float	1.3	Angular-rate feedforward gain
<code>max_angular_z</code>	float	0.12 rad/s	Maximum angular velocity

Distance-follow parameters (3D pose Z-axis):

Parameter	Type	Default Value	Description
<code>enable_distance_follow</code>	bool	true	Whether to enable distance following
<code>target_tag_pose_z</code>	float	0.35 m	Distance-follow target Z-axis depth
<code>distance_kp</code>	float	0.6	3D distance PD proportional gain (high gain)
<code>distance_kd</code>	float	0.2	3D distance PD derivative gain
<code>max_linear_x</code>	float	0.12 m/s	Maximum linear velocity
<code>distance_tolerance_pose_z</code>	float	0.06 m	Distance deadband

AprilTag detection parameters:

Parameter	Type	Default Value	Description
<code>tag_family</code>	str	tag36h11	AprilTag tag family
<code>tag_size</code>	float	0.048 m	Tag physical side length
<code>quad_decimate</code>	float	2.0	Quad-tree downsampling factor (≥ 1.0)
<code>tag_select_strategy</code>	str	largest	Multi-Tag selection strategy (largest / smallest_id)
<code>camera_yaml_path</code>	str	.../config/camera.yaml	Camera intrinsics file path
<code>use_camera_yaml</code>	bool	true	Whether to load the camera calibration file

6. Frequently Asked Questions

Problem	Cause	Solution
Inaccurate distance following	The actual Tag size does not match the <code>tag_size</code> parameter	Verify that the printed Tag size matches the parameter (default 0.048 m)
Pose jitter	The Tag is too small or the lighting is insufficient	Increase the Tag size and improve the lighting
Small Tag not detected	<code>quad_decimate=2.0</code> downsampling too aggressive	Reduce <code>quad_decimate</code> (e.g., 1.0)
dt_apriltags not installed	Missing Python package	<code>pip install dt-apriltags</code>
Degraded to pixel following	The camera calibration file <code>camera.yaml</code> does not exist	Run the camera calibration to generate the intrinsics file